

SKALA Python 데이터 분석 실습 결과 보고서

4기_광주_3반_채수아

목차

1부. Python 실습

실습 1. 대용량 웹 로그 스트리밍 집계

실습 2. Pydantic v2 중첩 스키마 검증

실습 3. asyncio 기반 비동기 데이터 수집

2부. 종합 실습

종합실습 1. 비동기 ETL 파이프라인

추가과제. ETL 블랙박스

결과 및 자기 평가

1부. Python 실습

1. 실습 1 — 대용량 웹 로그 스트리밍 집계

200,000건의 웹 로그를 한 번에 메모리에 올리지 않고 한 행씩 읽어 요청 수, 상태코드, 경로, 시간대, 접속 IP를 집계했습니다.

1.1 구현 목표

- csv.DictReader와 제너레이터를 사용해 입력 행을 순차적으로 처리합니다.
- reduce와 Counter로 상태코드·경로·시간대·IP 집계를 한 번의 순회에서 수행합니다.
- 입력 파일 누락과 필수 컬럼 누락을 구분하여 원인을 포함한 오류를 출력합니다.
- 전체 로딩 방식과 스트리밍 방식의 최대 메모리를 tracemalloc로 비교합니다.

1.2 실행 방법

```
python ex01_streaming_agg/solution.py
```

```
python ex01_streaming_agg/memory_comparison.py
```

```
실습 1: 대용량 웹 로그 스트리밍 집계
=====
총 요청 수 : 200,000건
5xx 오류   : 16,047건
5xx 오류율 : 8.02%

[상태코드별 요청]
200: 159,884건
301: 8,055건
302: 8,007건
404: 8,007건
500: 8,126건
503: 7,921건

[인기 경로 TOP 5]
/static/app.js      20,204건
/health             20,152건
/checkout            20,128건
/cart                20,036건
/products/detail    20,030건

[시간대별 요청]
00시: 8,685건
01시: 8,543건
02시: 8,569건
03시: 8,769건
04시: 8,608건
05시: 8,515건
06시: 8,599건
07시: 8,674건
08시: 8,564건
09시: 8,729건
10시: 8,615건
11시: 8,633건
12시: 8,595건
13시: 8,607건
14시: 8,624건
15시: 8,672건
16시: 8,653건
17시: 8,613건
18시: 8,642건
19시: 7,100건
20시: 7,275건
21시: 7,203건
22시: 7,177건
23시: 7,336건

[접속 상위 IP TOP 5]
83.3.180.55      2건
173.10.79.216    2건
84.136.62.63     2건
205.212.216.202  2건
186.215.89.61    2건
INFO | 로그 집계를 완료했습니다: 총 200,000건
```

실습 1 — 웹 로그 스트리밍 집계 실행 결과

1.3 실행 결과

전체 요청: 200,000건

5xx 오류: 16,047건 / 8.02%

인기 경로 TOP 3: /static/app.js 20,204건, /health 20,152건, /checkout 20,128건

상태코드: 200 159,884건, 301 8,055건, 302 8,007건, 404 8,007건, 500 8,126건, 503 7,921건

1.4 메모리 비교

동일한 200,000건을 처리했을 때 전체 로딩 방식은 24.61MB, 스트리밍 방식은 0.15MB의 최대 메모리를 사용했습니다. 측정 환경 기준으로 스트리밍 방식이 약 162.8배 적은 메모리를 사용해 대용량 파일 처리 목적에 적합함을 확인했습니다.

```
=====
실습 1 확장 : 로그 읽기 방식별 최대 메모리 비교
=====
전체 로딩 : 200,000건 / 24.61 MB
스트리밍 : 200,000건 / 0.15 MB
메모리 차이 : 162.8배
INFO | 두 방식 모두 200000건을 처리했습니다.
```

실습 1 — 로그 읽기 방식별 최대 메모리 비교

1.5 확인한 점

- 대용량 CSV는 리스트 전체 로딩보다 제너레이터 기반 순차 처리가 효율적입니다.
- Counter를 사용하면 여러 기준의 빈도 집계를 간결하게 구성할 수 있습니다.
- 한 행에서 필요한 값을 모두 누적해 같은 데이터를 반복해서 읽지 않도록 했습니다.

2. 실습 2 — Pydantic v2 중첩 스키마 검증

JSON API 응답 40건을 Pydantic v2 모델로 검증하고, 유효 데이터와 오염 데이터를 분리하여 각각 UTF-8 JSON 파일로 저장했습니다.

2.1 검증 구조

- UserRecord 모델에서 ID, 사용자명, 이메일, 나이, 가입일, 태그를 검증합니다.
- Profile 중첩 모델에서 국가, 회원 등급, 점수 범위를 검증합니다.
- 이메일은 앞뒤 공백 제거와 소문자 변환 후 정규식으로 형식을 확인합니다.
- 한 레코드의 실패가 전체 검증을 중단하지 않도록 건별로 ValidationError를 수집합니다.

2.2 실행 방법

```
python ex02_pydantic/solution.py
```

```
INFO | API 응답 검증을 시작합니다: C:\Users\teo\skala-assignment-upload\Python_data_0720_Day1\source\data\api_response.json
=====
실습 2: Pydantic v2 중첩 스키마 검증
=====
전체 40건 → 유효 36건 / 오염 4건

[오염 데이터 상세]
행   ID   필드   사유
-----
7     7     age    Input should be greater than or equal to 0
13    13    email  Value error, 올바른 이메일 형식이 아닙니다
21    21    email  Field required
29    29    profile.score  Input should be less than or equal to 100
INFO | 검증을 완료했습니다: 유효 36건, 오염 4건
INFO | 검증 결과를 저장했습니다: C:\Users\teo\skala-assignment-upload\Python_data_0720_Day1\source\ex02_pydantic\output
```

실습 2 — Pydantic 중첩 스키마 검증 결과

2.3 실행 결과

전체 데이터: 40건

유효 데이터: 36건

오염 데이터: 4건

행 7 / ID 7: age가 -5로 허용 범위(0~120)를 벗어남

행 13 / ID 13: email이 올바른 이메일 형식이 아님

행 21 / ID 21: 필수 필드 email이 누락됨

행 29 / ID 29: profile.score가 150으로 최대값 100을 초과함

2.4 생성 파일

- output/valid_records.json — 검증을 통과한 36건
- output/invalid_records.json — 원본 데이터와 필드별 오류 정보를 포함한 4건

```
[
  {
    "row": 7,
    "id": 7,
    "data": {
      "id": 7,
      "username": "user_007",
      "email": "user7@example.com",
      "age": -5,
      "is_active": true,
      "signup_date": "2024-11-07",
      "profile": {
        "country": "DE",
        "tier": "free",
        "score": 87.43
      },
      "tags": [
        "web"
      ]
    },
    "errors": [
      {
        "type": "greater_than_equal",
        "loc": [
          "age"
        ],
        "msg": "Input should be greater than or equal to 0",
        "input": -5,
        "ctx": {
          "ge": 0
        },
        "url": "https://errors.pydantic.dev/2.13/v/greater_than_equal"
      }
    ]
  },
]
```

실습 2 — 오염 데이터 분리 파일 확인

2.5 확인한 점

- 중첩 모델을 사용하면 JSON 구조와 업무 규칙을 코드에서 명확하게 표현할 수 있습니다.
- 오류 위치와 원본 데이터를 함께 보존하면 재처리와 원인 분석이 쉬워집니다.
- 검증과 정규화를 모델에 모아 후속 처리에는 일관된 데이터만 전달할 수 있습니다.

3. 실습 3 — asyncio 기반 비동기 데이터 수집

60건의 모의 API 요청을 asyncio로 동시에 처리하고, 동시성 제한·타임아웃·지수 백오프 재시도·최종 실패 격리를 포함한 비동기 수집기를 구현했습니다.

3.1 구현 구조

- asyncio.Semaphore로 동시에 실행되는 요청을 최대 10건으로 제한합니다.
- asyncio.wait_for로 요청별 제한 시간을 적용합니다.
- 일시 오류 발생 시 최대 3회까지 지수 백오프로 재시도합니다.
- asyncio.gather(return_exceptions=True)로 일부 실패가 전체 작업을 중단하지 않게 합니다.
- 성공 결과, 최종 실패, 실행 지표를 각각 JSON 파일로 저장합니다.

3.2 실행 방법

```
python ex03_async_collector/solution.py
```

```
INFO | 비동기 수집을 시작합니다: 요청 60건, 동시 제한 10건
WARNING | ID 17 수집 실패 (1/3): 일시적인 모의 서버 오류 - 0.05초 후 재시도
WARNING | ID 41 수집 실패 (1/3): 일시적인 모의 서버 오류 - 0.05초 후 재시도
=====
실습 3: asyncio 기반 비동기 수집기
=====
요청 수           : 60건
동시 요청 제한    : 10건
성공 / 최종 실패  : 60건 / 0건
재시도           : 2회 {17: 1, 41: 1}
비동기 실행 시간   : 1.74초
동기 실행 시간     : 14.42초
실측 속도 향상    : 약 8.3배
실행 모드         : 오프라인 모의 실행
결과 저장 위치    : C:\Users\teo\skala-assignment-upload\Python_data_0720_Day1\source\ex03_async_collector\output
INFO | 비동기 수집과 결과 저장을 완료했습니다.
```

실습 3 — 동시 수집과 재시도 실행 결과

3.3 성능 및 안전성 결과

요청 수: 60건

동시 요청 제한: 10건

성공 / 최종 실패: 60건 / 0건

재시도: 2회 — ID 17, ID 41이 각 1회 후 성공

비동기 실행 시간: 약 1.75초

동기 실행 시간: 약 14.43초

실측 속도 향상: 약 8.2배

3.4 생성 파일

- output/collected.json — 수집에 성공한 60건

- output/dead_letter.json — 최종 실패 목록(이번 실행은 빈 배열)
- output/metrics.json — 요청 수, 재시도, 실행 시간, 동시성 설정

```
{
  "requested": 60,
  "collected": 60,
  "failed": 0,
  "retries": 2,
  "retry_ids": {
    "17": 1,
    "41": 1
  },
  "sequential_seconds": 14.4207,
  "elapsed_seconds": 1.7442,
  "max_concurrent": 10,
  "max_attempts": 3,
  "mode": "mock"
}
```

실습 3 — 비동기 수집 실행 지표

3.5 확인한 점

- 네트워크 대기 작업은 비동기 처리로 전체 소요 시간을 크게 줄일 수 있습니다.
- Semaphore는 외부 서버와 로컬 자원을 보호하는 백프레셔 역할을 합니다.
- 재시도 중에는 Semaphore를 반납해 다른 요청의 진행을 막지 않도록 구성했습니다.

2부. 종합 실습

1. 종합실습 1 — 비동기 ETL 파이프라인

비동기 상품 수집(Extract), Pydantic 검증 및 정규화(Transform), CSV-Parquet 저장(Load)을 하나의 파이프라인으로 연결했습니다.

1.1 파이프라인 구성

- Extract — 60개 상품 ID를 최대 동시성 10으로 수집하고 일시 오류를 재시도합니다.
- Transform — Product와 Seller 중첩 모델로 타입과 업무 규칙을 검증합니다.
- Normalize — 상품명·지역의 공백을 제거하고 카테고리를 소문자로 통일합니다.
- Load — 중첩 데이터를 평면 DataFrame으로 변환해 CSV와 Parquet으로 저장합니다.
- Observe — 단계별 건수, 실패 정보, 실행 시간을 JSON으로 기록합니다.

1.2 실행 방법

```
python capstone01_async_etl/pipeline.py
```

```
INFO | 비동기 ETL 파이프라인을 시작합니다.
WARNING | 상품 ID 9 수집 실패 (1/3): 일시적인 모의 수집 지연 - 0.01초 후 재시도
WARNING | 상품 ID 37 수집 실패 (1/3): 일시적인 모의 수집 지연 - 0.01초 후 재시도
=====
종합실습 1: 비동기 ETL 파이프라인
=====
요청 / 수집 성공 : 60건 / 60건
수집 최종 실패 : 0건
검증 유효 / 무효 : 58건 / 2건
최종 저장 : 58건
실행 시간 : 0.36초
CSV : C:\Users\teo\skala-assignment-upload\Python_data_0720_Day1\source\capstone01_async_etl\output\products.csv
Parquet : C:\Users\teo\skala-assignment-upload\Python_data_0720_Day1\source\capstone01_async_etl\output\products.parquet
uet

요약 딕셔너리: {'requested': 60, 'extracted': 60, 'extract_failed': 0, 'valid': 58, 'transform_invalid': 2, 'invalid_total': 2, 'rows_saved': 58, 'elapsed_seconds': 0.3612, 'csv': 'C:\\Users\\teo\\skala-assignment-upload\\Python_data_0720_Day1\\source\\capstone01_async_etl\\output\\products.csv', 'parquet': 'C:\\Users\\teo\\skala-assignment-upload\\Python_data_0720_Day1\\source\\capstone01_async_etl\\output\\products.parquet'}
INFO | 비동기 ETL 파이프라인을 완료했습니다.
```

종합실습 1 — 비동기 ETL 파이프라인 실행 결과

1.3 단계별 실행 결과

요청: 60건

수집 성공 / 최종 실패: 60건 / 0건

검증 유효 / 무효: 58건 / 2건

최종 저장: 58건

실행 시간: 약 0.30초

검증 실패 ID: 13, 29 — price가 -100.0으로 0보다 커야 하는 규칙 위반

1.4 생성 산출물

- output/products.csv — UTF-8 형식의 분석.확인용 데이터
- output/products.parquet — 컬럼 타입을 보존하는 후속 처리용 데이터
- output/invalid_records.json — Extract-Transform 실패를 함께 기록
- output/summary.json — 단계별 처리 건수, 실행 시간, 저장 경로

```
1 id,name,category,price,seller_id,seller_region
2 1,Product 001,cloud,9.87,2,Seoul
3 2,Product 002,ai,11.24,3,Busan
4 3,Product 003,data,12.61,4,Gwangju
5 4,Product 004,cloud,13.98,5,Seoul
6 5,Product 005,ai,15.35,6,Busan
7 6,Product 006,data,16.72,7,Gwangju
8 7,Product 007,cloud,18.09,1,Seoul
9 8,Product 008,ai,19.46,2,Busan
10 9,Product 009,data,20.83,3,Gwangju
11 10,Product 010,cloud,22.2,4,Seoul
12 11,Product 011,ai,23.57,5,Busan
13 12,Product 012,data,24.94,6,Gwangju
14 14,Product 014,ai,27.68,1,Busan
15 15,Product 015,data,29.05,2,Gwangju
16 16,Product 016,cloud,30.42,3,Seoul
17 17,Product 017,ai,31.79,4,Busan
18 18,Product 018,data,33.16,5,Gwangju
19 19,Product 019,cloud,34.53,6,Seoul
20 20,Product 020,ai,35.9,7,Busan
21 21,Product 021,data,37.27,1,Gwangju

1 {
2   "requested": 60,
3   "extracted": 60,
4   "extract_failed": 0,
5   "valid": 58,
6   "transform_invalid": 2,
7   "invalid_total": 2,
8   "rows_saved": 58,
9   "elapsed_seconds": 0.3612,
0   "csv": "C:\\Users\\teo\\skala-assignment-upload\\Python_data_0720_Day1\\source\\capstone01_async_etl\\output\\products.csv",
1   "parquet": "C:\\Users\\teo\\skala-assignment-upload\\Python_data_0720_Day1\\source\\capstone01_async_etl\\output\\products.parquet"
2 }
```

1.5 테스트

스키마 정규화, 음수 가격 거부, 유효·무효 건수 보존, Parquet 왕복, 비동기 재시도, 전체 파이프라인 산출물을 6개 테스트로 검증했습니다.

- 카테고리 소문자 변환과 문자열 공백 제거
- 0 이하 가격의 검증 실패 처리
- 유효·무효 합계와 입력 건수의 일치
- Parquet 저장 후 DataFrame 동일성
- 일시 오류 ID의 재시도 후 전체 수집
- 30건 입력 기준 28건 저장과 실패 ID 13·29 분리

```
python -m pytest capstone01_async_etl/test_pipeline.py -v
```

```
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\teo\skala-assignment-upload\Python_data_0720_Day1\source\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\teo\skala-assignment-upload\Python_data_0720_Day1\source
plugins: anyio-4.14.2, cov-7.1.0
collected 6 items

capstone01_async_etl/test_pipeline.py::test_category_is_normalized_to_lowercase PASSED [ 16%]
capstone01_async_etl/test_pipeline.py::test_negative_price_is_rejected PASSED [ 33%]
capstone01_async_etl/test_pipeline.py::test_valid_and_invalid_counts_match_input PASSED [ 50%]
capstone01_async_etl/test_pipeline.py::test_parquet_round_trip PASSED [ 66%]
capstone01_async_etl/test_pipeline.py::test_extract_collects_all_ids_after_retry PASSED [ 83%]
capstone01_async_etl/test_pipeline.py::test_run_summary_and_outputs PASSED [100%]

===== 6 passed in 1.14s =====
```

종합실습 1 — 자동화 테스트 6건 통과

1.6 확인한 점

- ETL 단계를 함수로 분리해 각 단계의 책임과 테스트 범위를 명확히 했습니다.
- 검증 실패 데이터를 버리지 않고 별도 저장해 재처리 가능성을 확보했습니다.
- CSV와 Parquet을 함께 제공해 사람이 확인하기 쉬운 형식과 타입 보존 형식을 모두 지원했습니다.

추가과제. ETL 블랙박스

정상·지연·장애 폭주 상황을 동일한 조건으로 재현하고, 고정 동시성과 적응형 동시성 제어가 ETL 안정성에 미치는 영향을 비교했습니다. 각 요청의 재시도와 지연, 최종 실패를 이벤트 로그로 남겨 결과를 다시 추적할 수 있게 구성했습니다.

1. 과제 목표

- 고정 시드 기반 모의 API로 같은 장애 조건을 반복 재현합니다.
- 고정 동시성 3·10·30과 적응형 제어를 같은 60건 입력으로 비교합니다.
- 타임아웃, 지수 백오프 재시도, 최종 실패 격리까지 하나의 비동기 ETL 흐름에 적용합니다.
- JSONL 이벤트 로그, CSV 결과, JSON 요약, HTML 리포트를 실행마다 함께 생성합니다.

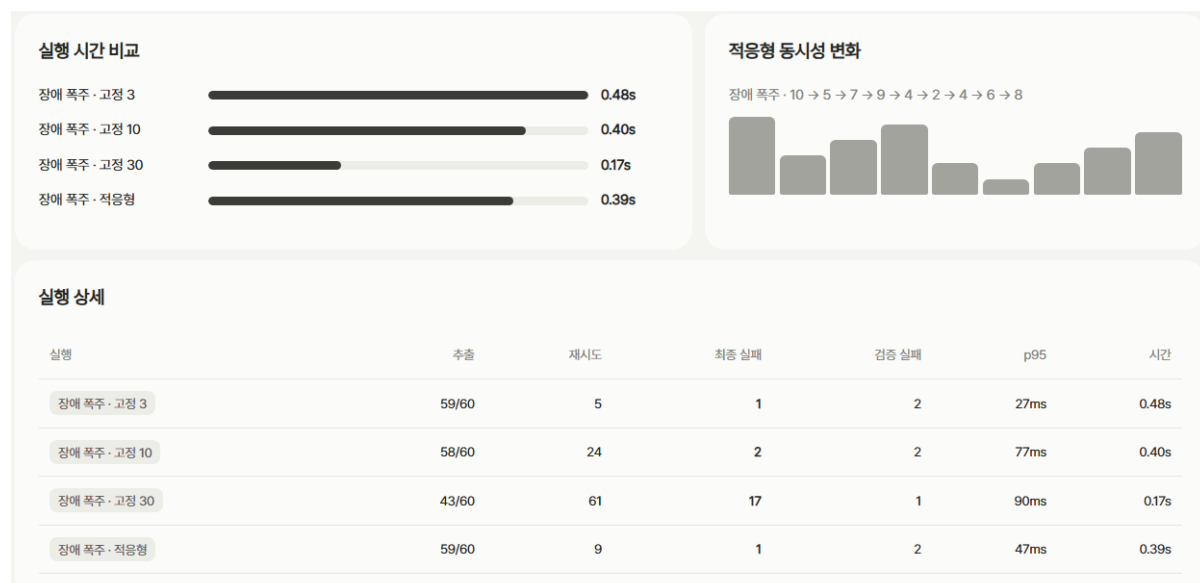
2. 실험 설계

정상, 간헐적 지연, 동시 요청이 몰릴 때 오류율이 높아지는 장애 폭주 시나리오를 만들었습니다. 적응형 모드는 최근 응답의 p95 지연과 오류율을 관찰해 동시성을 2~20 범위에서 조정합니다.

```
python Advanced/etl_blackbox/main.py
```

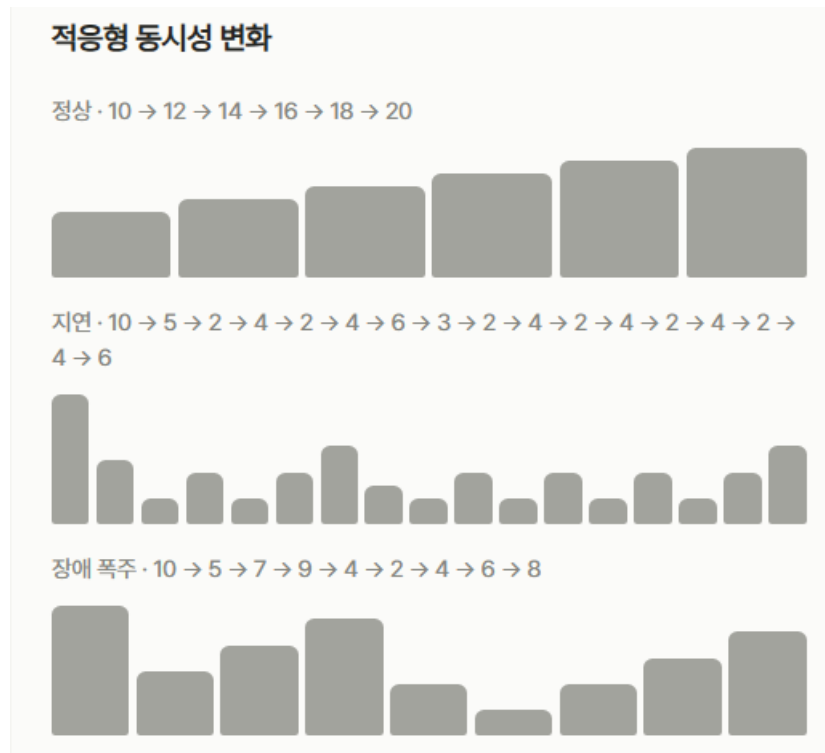
3. 장애 폭주 실행 결과

고정 30은 0.168초로 가장 빨랐지만 17건이 최종 실패했고 재시도도 61회 발생했습니다. 적응형은 0.403초가 걸렸지만 59건을 수집해 최종 실패를 1건으로 줄였고, 재시도는 9회에 그쳤습니다.



4. 적응형 제어 동작

정상 시나리오에서는 동시성이 10에서 20까지 완만하게 증가했습니다. 지연과 장애 폭주가 감지된 구간에서는 10에서 5 이하로 즉시 낮춘 뒤, 응답이 안정되면 2건씩 다시 늘려 처리량과 안정성의 균형을 맞췄습니다.



추가과제 — 시나리오별 적응형 동시성 변화

5. 생성 산출물과 검증

- output/runs/* — 요청 이벤트 JSONL과 검증 통과 레코드 CSV
- experiment_summary.json·etl_blackbox_report.html — 12개 실험 비교 결과
- pytest 6건 통과 — 백오프, 실패 격리, 적응형 증감, 산출물 생성을 자동 검증

```
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\teo\skala-assignment-upload\Python_data_0720_Day1\source\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\teo\skala-assignment-upload\Python_data_0720_Day1\source
plugins: anyio-4.14.2, cov-7.1.0
collected 6 items

Advanced/etl_blackbox/test_blackbox.py::test_normal_scenario_collects_every_item PASSED [ 16%]
Advanced/etl_blackbox/test_blackbox.py::test_permanent_failures_move_to_dead_letter PASSED [ 33%]
Advanced/etl_blackbox/test_blackbox.py::test_adaptive_mode_reduces_concurrency_during_outage PASSED [ 50%]
Advanced/etl_blackbox/test_blackbox.py::test_invalid_price_is_rejected_by_transform PASSED [ 66%]
Advanced/etl_blackbox/test_blackbox.py::test_same_seed_produces_same_outcome_counts PASSED [ 83%]
Advanced/etl_blackbox/test_blackbox.py::test_jsonl_csv_and_html_outputs_are_created PASSED [100%]

===== 6 passed in 0.91s =====
```

추가과제 — 자동화 테스트 6건 통과

결과 및 자기 평가

1. 전체 결과

실습 1부터 추가과제까지 입력 검증, 메모리 효율, 비동기 동시성, 실패 격리, 데이터 정규화와 다중 형식 저장을 단계적으로 구현했습니다. 추가과제에서는 정상·지연·장애 폭주 조건을 반복 실행해 처리 속도와 안정성의 차이를 수치로 비교했습니다.

2. 잘된 점

- 20만 건 로그를 한 번만 순회하면서 여러 기준을 동시에 집계했습니다.
- Pydantic 중첩 모델로 필드 타입, 값 범위, 필수값과 사용자 정의 규칙을 함께 검증했습니다.
- 비동기 수집에 동시성 제한, 타임아웃, 재시도와 dead-letter 분리를 적용했습니다.
- 종합실습에서 Extract·Transform·Load를 독립 함수로 분리하고 6개 자동화 테스트를 통과했습니다.
- CSV, Parquet, JSON을 목적에 맞게 구분해 저장하고 처리 요약을 함께 남겼습니다.
- 장애 폭주 실험에서 적응형 제어가 고정 30 대비 최종 실패를 17건에서 1건으로, 재시도를 61회에서 9회로 줄이는 것을 확인했습니다.

3. 보완할 점

- 현재 실험은 재현 가능한 모의 API를 사용하므로 실제 API의 요청 제한, 지역별 네트워크 편차, 장시간 장애 패턴을 추가 검증할 필요가 있습니다.
- HTML 리포트를 외부 모니터링 시스템과 연결하고 실패율 임계치 알림을 추가하면 운영 중 이상 징후를 더 빠르게 파악할 수 있습니다.
- 대용량 데이터가 더 커질 경우 청크 단위 Parquet 저장과 증분 처리 전략을 추가할 수 있습니다.
- 재처리 대상의 우선순위와 최대 보관 기간을 정의하면 실패 데이터 운영이 더 체계적이 됩니다.

4. 종합 평가

이번 실습을 통해 데이터 처리 성능은 실행 속도뿐 아니라 메모리 사용량, 입력 신뢰성, 부분 실패 복구와 결과 추적 가능성을 함께 고려해야 함을 확인했습니다. 특히 장애 폭주 실험은 가장 빠른 설정이 항상 가장 안전한 설정은 아니라는 점을 보여주었고, 응답 상태에 따라 동시성을 조정하는 방식이 안정적인 ETL 운영에 효과적임을 확인했습니다.